

732A94 Advanced R Programming

Computer lab 5: API R projects

Johan Alenlöv, Bayu Brahmantio.

Below are some API that can be used as a part of the assignment to implement an R package to connect to an API. In connection to each API there is a short description of the data, some ideas of what could be done with it, and the difficulty of implementation is indicated using one, two or three stars with one star (simpler) to three stars (difficult).

All APIs below are free to use for small, non-commercial projects. Some of them require a (free) API key. If you use an API with a key, the key must **never** be committed to GitHub. Read it from an environment variable instead (e.g. `Sys.getenv("MY_API_KEY")`) and think about how your unit tests and CI should behave when no key is available. Also respect the rate limits of the API, both in your package and in your test suite.

In general there exist many different web APIs and you can choose any API you may find interesting. If you choose any other API than the ones listed below, you need to confirm your choice with the teacher. The only restriction is that the API should not already have an R package.

APIs with geographical data

These APIs return data with coordinates, which makes them well suited for maps. The `leaflet` R package can draw interactive maps using OpenStreetMap tiles without any API key, and works well together with Shiny.

*** USGS Earthquake API** The U.S. Geological Survey provides a catalogue of all recorded earthquakes in the world through the FDSN event web service. No API key is needed. Each earthquake comes with time, latitude, longitude, depth, magnitude, a place description, a tsunami flag, the number of “felt” reports and an alert level. Queries can be restricted by time window, magnitude, a bounding box or a radius around a point, and the results are returned as GeoJSON or CSV (at most 20 000 events per request). More information can be found [here](https://earthquake.usgs.gov/fdsnws/event/1/) (<https://earthquake.usgs.gov/fdsnws/event/1/>).

Ideas: a function that takes a time period, a region and a minimum magnitude and returns a tidy `data.frame`; a Shiny application with a map where the size of the circles shows the magnitude and a slider for the time period; plots of the number of earthquakes per day or the distribution of magnitudes.

*** CityBikes** CityBikes collects live data from more than 800 bike sharing systems around the world, among them Malmö, Lund and Göteborg. No API key is needed (the rate limit is 300 requests per hour). The API has two levels: a list of all networks with city, country and coordinates, and for each network a list of stations with coordinates, the number of free bikes and empty slots and a timestamp. More information can be found [here](https://api.citybik.es/v2/) (<https://api.citybik.es/v2/>).

Ideas: functions to find networks by city or country and to get the stations of a network; a Shiny application where the user chooses a city and gets a map showing where there are bikes available right now; compare the size of the bike systems in different countries.

**** NASA FIRMS – active fires** NASA’s Fire Information for Resource Management System (FIRMS) provides near real-time fire detections from the MODIS and VIIRS satellites. A free `MAP_KEY` is needed (obtained by e-mail). Each detected fire comes with latitude, longitude, brightness, fire radiative power, confidence, acquisition date and time and whether it was detected during day or night. Data can be requested for a country or a bounding box for the last 1–10 days and is returned as CSV. More information

can be found [here](#)

(<https://firms.modaps.eosdis.nasa.gov/api/>).

Ideas: a function returning all fires in a given country during the last days; a map of the fires coloured by fire radiative power; compare the number of fires between countries or between day and night.

**** Open Charge Map** Open Charge Map is a global register of charging stations for electric vehicles. A free API key is needed. Each charging location comes with address, coordinates, operator, the number of charging points, connector types, power (kW), status and usage cost. Locations can be searched by country, bounding box or distance from a point, and filtered on e.g. connector type or minimum power. The JSON is nested and uses reference tables (operators, connection types, etc.), so some work is needed to create tidy data. More information can be found [here](#)

(<https://openchargemap.org/site/develop/api>).

Ideas: a function to find chargers close to a given point; a map of all fast chargers along a route (e.g. the E4); compare the number of chargers per operator or connector type between countries.

**** Trafiklab – ResRobot** Trafiklab gives access to data for public transport in Sweden. The ResRobot API covers all operators in Sweden and requires a free API key (with a monthly quota). The documentation is in English. It contains stop lookup (by name or near a coordinate) with coordinates, real-time departure and arrival boards for a stop and a travel planner from A to B, including the intermediate stops of a trip. More information can be found [here](#)

(<https://www.trafiklab.se/api/our-apis/resrobot-v21/>).

Ideas: functions for finding stops and getting the next departures; a Shiny application showing a departure board for a chosen stop; plot a planned trip as a route on a map.

Other APIs

*** Nobel Prize API** The Nobel Prize organisation provides data about all Nobel Prizes and laureates. No API key is needed. For each laureate there is information about name, gender, birth and death date, birth place (city and country, both historical and present-day name), affiliations and the motivation for the prize. For each prize there is year, category and prize amount. More information can be found [here](#) (<https://www.nobelprize.org/about/developer-zone-2/>).

Ideas: functions for getting laureates by category and year; the age of laureates over time; a map of the birth countries of the laureates; a Shiny application to explore the laureates of a chosen category.

**** The Metropolitan Museum of Art Collection API** The Met in New York provides data on about 470 000 objects in its collection. No API key is needed, but the rate limit is 80 requests per second. Each object has title, artist, date, medium, dimensions, culture, country, department, a public domain flag and links to images. A search only returns object IDs, so the package needs to fetch each object separately, which makes it important to handle many requests in a good way. More information can be found [here](#) (<https://metmuseum.github.io/>).

Ideas: a function that searches for e.g. an artist and returns a `data.frame` with the objects; a Shiny application showing the images of the search result; the distribution of the objects over time for different departments.

*** PokéAPI** PokéAPI contains data about all Pokémon and the Pokémon games. No API key is needed, but the users should cache results and not overuse the API. Each Pokémon has stats (HP, attack, defense, etc.), types, abilities, moves, height, weight and images (sprites). There is also data on species, evolution chains, items and in-game locations. The JSON is deeply nested and lists are paginated. More information can be found [here](#) (<https://pokeapi.co/docs/v2>).

Ideas: functions for getting the stats of one or several Pokémon as a `data.frame`; compare the stats of different types; a Shiny application where the user chooses two Pokémon and compares them.

*** Open Trivia Database** Open Trivia DB is a free database with about 5 000 verified quiz questions. No API key is needed. Each question has a category, a difficulty, a type (multiple choice or true/false), the correct answer and the incorrect answers. It is possible to get the number of questions per category and difficulty. The API uses session tokens to avoid getting the same question twice, allows only one

request per 5 seconds from the same IP address and returns special characters encoded (HTML, URL or base64 encoding). More information can be found [here](https://opentdb.com/api_config.php) (https://opentdb.com/api_config.php).

Ideas: a function to get questions as a **data.frame** that handles tokens, decoding and the rate limit; a quiz game in Shiny where the user chooses category and difficulty; plot the number of questions per category and difficulty.